

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1703

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

**Duration : 1 Hour
Total Marks:50**

Annexure A

Reverse Engineering

Code of Conduct:

*I K. Pujitha (192524149) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: K. Pujitha

Annexure A

Reverse Engineering

1. Reverse Engineering a Recommendation System

A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

2. Reverse Engineering a Smart Traffic Management System

A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data

pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

3. Reverse Engineering a Fraud Detection Pipeline

A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

4. Reverse Engineering a Personalized Learning System

An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution

		industry standards			
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

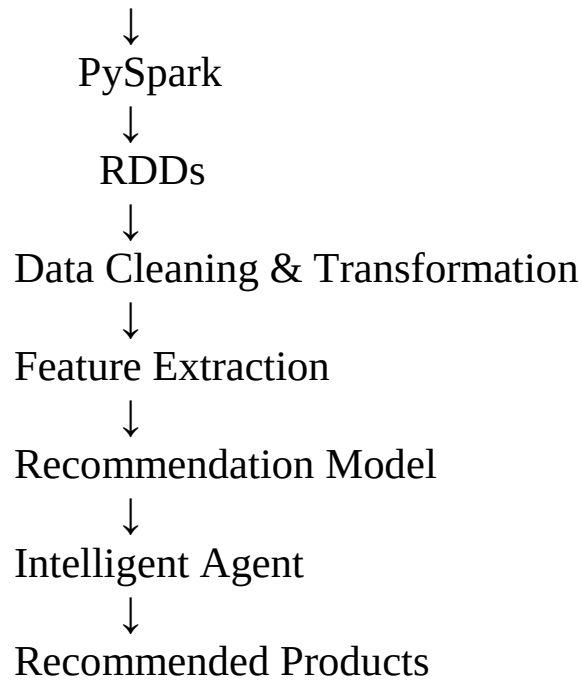
1. Reverse Engineering a Recommendation System

Given behavior

An e-commerce website gives highly accurate product recommendations in real time.

What we infer

User Activity
(clicks, searches, purchases)



RDD operations

For example:

```
data = sc.parallelize([
    ("U1", "Phone"),
    ("U1", "Laptop"),
    ("U2", "Phone")
])
```

```
items = data.map(lambda x: (x[0], x[1]))
```

```
result = items.collect()
print(result)
```

Important operations to mention:

- `map()` → transform user/item data
- `filter()` → remove unwanted data
- `reduceByKey()` → aggregate user activity
- `groupByKey()` → group items by user

- join() → combine different datasets

Intelligent Agent

The Recommendation Agent observes:

User searches Laptop



Agent updates user preference



Finds related products



Updates recommendations

Reverse-engineering conclusion

The observed real-time recommendations suggest a pipeline where user behavior is continuously collected, distributed through PySpark, transformed using RDDs, processed by recommendation algorithms, and dynamically updated by intelligent agents.

2. Reverse Engineering a Smart Traffic Management System

Given behavior

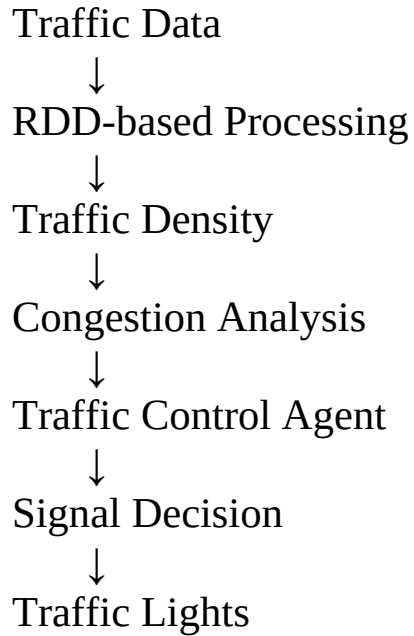
Traffic signals automatically change according to:

- Vehicle density
- Congestion
- Traffic patterns

Reconstructed architecture

Cameras / Sensors





Example

Road A → 80 vehicles

Road B → 15 vehicles

Road C → 30 vehicles

The agent may infer:

Road A → Give longer GREEN time

Road B → Give shorter GREEN time

Road C → Normal GREEN time

Intelligent Agents

Traffic Monitoring Agent

→ collects vehicle density.

Traffic Analysis Agent

→ calculates congestion.

Signal Control Agent

→ changes signal timing.

Emergency Agent

→ gives priority to emergency vehicles.

Time constraint

Traffic decisions must be made very quickly because waiting too long can increase congestion.

Key point for exam

The system is reverse engineered by observing signal behavior and inferring that real-time sensor data is distributed using PySpark, processed using RDD operations, and supplied to intelligent agents that make time-sensitive signal decisions.

3. Reverse Engineering a Fraud Detection Pipeline

Given behavior

A bank detects suspicious transactions within milliseconds.

Reconstructed system

Transactions



PySpark



RDD



Data Cleaning



Feature Extraction



Anomaly Detection



Fraud Risk Score



Fraud Detection Agent



Alert / Block

Example features

The system may examine:

- Transaction amount
- Transaction location
- Transaction frequency
- Time of transaction
- User's normal behavior

Example:

Normal:

User usually spends ₹2,000

New transaction:

₹2,00,000 from another location



Anomaly detected



High fraud score



Alert

Agents

- **Transaction Agent** → monitors transactions
- **Anomaly Agent** → detects unusual patterns
- **Risk Agent** → calculates risk
- **Alert Agent** → sends warning

- **Learning Agent** → learns from confirmed fraud

Important reverse-engineering idea

Because the system responds within milliseconds, we infer that data processing and anomaly detection are highly distributed and optimized for low latency.

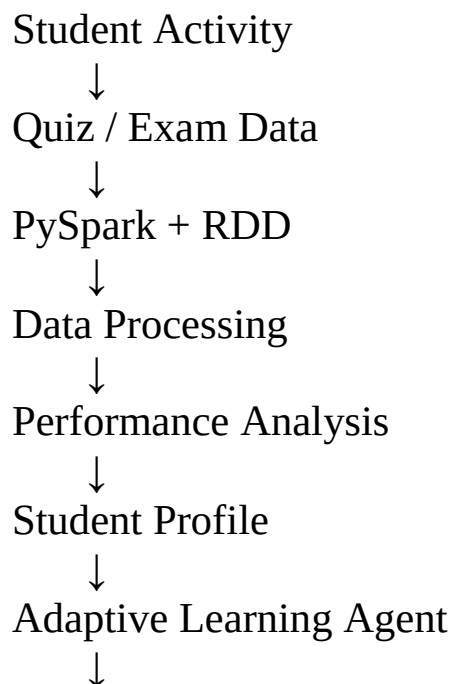
4. Reverse Engineering a Personalized Learning System

Given behavior

An online learning platform changes content difficulty based on:

- Student marks
- Learning speed
- Engagement
- Mistakes

Reconstructed architecture



Content Selection



Student



Feedback



Example

Student scores:

Math $\rightarrow$ 90%

Python $\rightarrow$ 45%

AI $\rightarrow$ 80%

The system may infer:

Math $\rightarrow$ Advanced content

Python $\rightarrow$ Basic content + practice

AI $\rightarrow$ Intermediate content

RDD example

```
scores = sc.parallelize([
    ("Math", 90),
    ("Python", 45),
    ("AI", 80)
])
```

```
weak = scores.Filter(lambda x: x[1] < 50)
```

```
print(weak.collect())
```

Output:

```
[('Python', 45)]
```

Intelligent Agents

Student Agent

→ maintains student profile.

Performance Agent

→ analyzes marks.

Content Agent

→ selects suitable material.

Feedback Agent

→ provides immediate feedback.

Adaptation Agent

→ changes difficulty.

5. Reverse Engineering a Cloud-Based AI Monitoring System

Given behavior

A cloud platform:

- Monitors system logs
- Detects anomalies
- Automatically scales resources
- Responds to failures

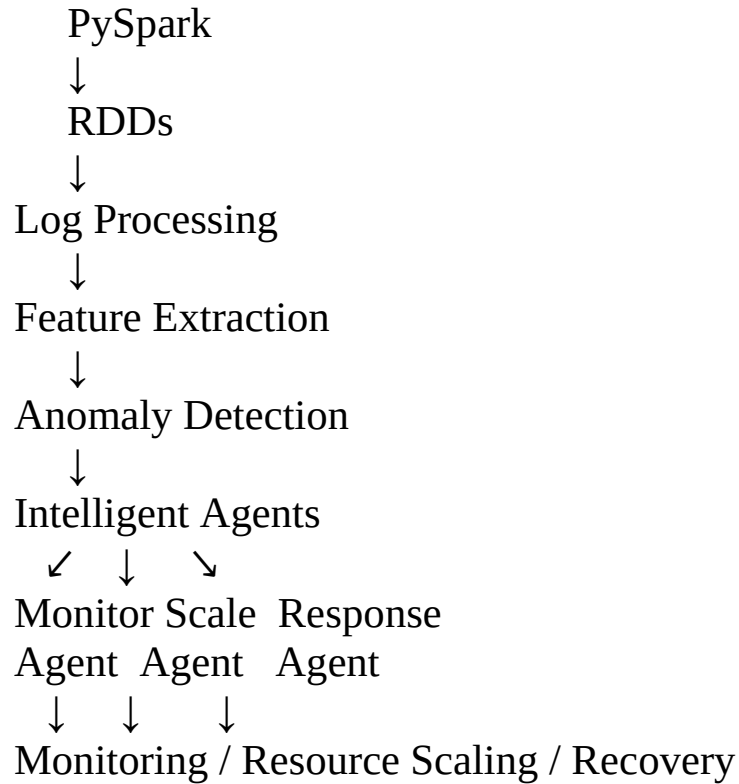
Reconstructed architecture

Cloud Servers



System Logs





RDD operations

For example:

```
logs = sc.parallelize([
    "server normal",
    "server normal",
    "server error",
    "server normal"
])

errors = logs.filter(
    lambda x: "error" in x
)

print(errors.collect())
```

Output:

['server error']

Intelligent Agents

Monitoring Agent

→ continuously checks system health.

Anomaly Agent

→ detects unusual behavior.

Scaling Agent

→ increases/decreases cloud resources.

Recovery Agent

→ responds to failures.

Example

CPU usage = 95%



Monitoring Agent



Scaling Agent



Add more resources



CPU usage decreases